MONASH University

ETC4500/ETC5450
Advanced R programming

Week 7: Measuring and improving performance

Profiling functions

- `Rprof()` : records every function call.
- `summaryRprof()` : summarises the results.
- `profvis()` : visualises the results.

Profiling

Where are the bottlenecks in your code?

```
library(profvis)
library(bench)
f <- function() {
  pause(0.1)
  g()
  h()
}
g <- function() {
  pause(0.1)
  h()
}
h <- function() {
```

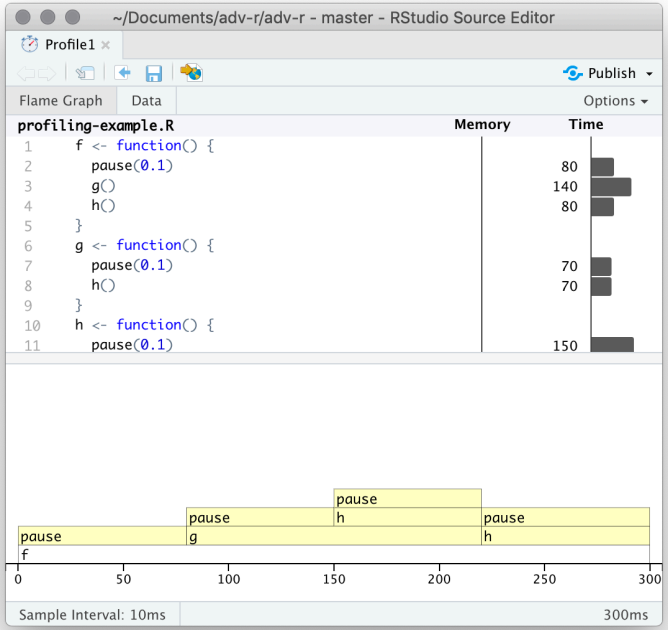
```
  pause(0.1)
}
```

Profiling

```r
tmp <- tempfile()
Rprof(tmp, interval = 0.1)
f()
Rprof(NULL)
writeLines(readLines(tmp))
#> sample.interval=100000
#> "pause" "g" "f"
#> "pause" "h" "g" "f"
#> "pause" "h" "f"
```

Profiling

```r
source(here::here("week4/profiling-example.R"))
profvis(f())
```


~/Documents/adv-r/adv-r - master - RStudio Source Editor
Profile1
Publish
Flame Graph
Data
Options
profiling-example.R
Memory
Time
f <- function() {
pause(0.1)
g()
h()
}
g <- function() {
pause(0.1)
h()
}
h <- function() {
pause(0.1)
80
140
80
70
70
150
pause
pause
h
pause
pause
g
h
f
0
50
100
150
200
250
300
Sample Interval: 10ms
300ms

Microbenchmarking

system.time()

```
x <- rnorm(1e6)
system.time(min(x))
```

```
   user  system elapsed
  0.001   0.000   0.001
```

```
system.time(sort(x)[1])
```

```
   user  system elapsed
  0.040   0.007   0.047
```

```r
system.time(x[order(x)[1]])
```

```
   user  system elapsed
  0.034   0.005   0.040
```

Microbenchmarking

bench::mark()

```
bench::mark(
  min(x),
  sort(x)[1],
  x[order(x)[1]]
)
```

```
# A tibble: 3 × 6
  expression          min   median `itr/sec` mem_alloc `gc/sec`
  <bch:expr>     <bch:tm> <bch:tm>     <dbl> <bch:byt>    <dbl>
1 min(x)          870.3µs  916.8µs    1075.         0B     0
2 sort(x)[1]       41.5ms     43ms      23.0   11.44MB    32.1
3 x[order(x)[1]]   32.8ms   33.5ms      29.8    3.81MB     4.58
```

Microbenchmarking

- `mem_alloc` tells you the memory allocated in the first run.
- `n_gc` tells you the total number of garbage collections over all runs.
- `n_itr` tells you how many times the expression was evaluated.
- Pay attention to the units!

Exercises

2. What's the fastest way to compute a square root? Compare:

   - `sqrt(x)`
   - `x^0.5`
   - `exp(log(x) / 2)`

   Use `system.time()` find the time for each operation.

   Repeat using `bench::mark()`. Why are they different?

Vectorization

- Vectorization is the process of converting a repeated operation into a vector operation.
- The loops in a vectorized function are implemented in C instead of R.
- Using `map()` or `apply()` is **not** vectorization.
- Matrix operations are vectorized, and usually very fast.

Exercises

Write the following algorithm to estimate $\int_0^1 x^2 dx$ using vectorized code

Monte Carlo Integration

a. Initialise: `hits = 0`
b. for i in 1:N
   - Generate two random numbers, U_1, U_2, between 0 and 1
   - If $U_2 < U_1^2$, then `hits = hits + 1`
c. end for
d. Area estimate = hits/N

Exercises

4. Use `bench::mark()` to compare the speed of `sq()` and `memo_sq()`.

Caching: using rds

```r
if (file.exists("results.rds")) {
  res <- readRDS("results.rds")
} else {
  res <- compute_it() # a time-consuming function
  saveRDS(res, "results.rds")
}
```

```r
res <- xfun::cache_rds(
  compute_it(), # a time-consuming function
  file = "results.rds"
)
```

Caching: using rds

```r
compute <- function(...) {
  xfun::cache_rds(rnorm(6), file = "results.rds", ...)
}
compute()
```

```
[1]  0.174  1.706  1.540 -2.640  1.819  0.834
```

```r
compute()
```

```
[1]  0.174  1.706  1.540 -2.640  1.819  0.834
```

```r
compute(rerun = TRUE)
```

```
[1] -2.003  1.300 -1.241  1.657  0.469 -0.790
```

```
compute()
```

```
[1] -2.003  1.300 -1.241  1.657  0.469 -0.790
```

Caching downloads

You often want to prevent downloads of the same data multiple times.

```
download_data <- function(url) {
  dest_folder <- tempdir()
  sanitized_url <- stringr::str_replace_all(url, "/", "_")
  dest_file <- file.path(dest_folder, paste0(sanitized_url, ".rds"))
  if (file.exists(dest_file)) {
    data <- readRDS(dest_file)
  } else {
    data <- read_tsv(url, show_col_types = FALSE)
    saveRDS(data, dest_file)
  }
```

```
  data
}
bulldozers <- download_data("https://robjhyndman.com/data/Bulldozers.
csv")
```

Caching: memoise

Caching stores results of computations so they can be reused.

```r
library(memoise)
sq <- function(x) {
  print("Computing square of 'x'")
  x**2
}
memo_sq <- memoise(sq)
memo_sq(2)
```

```
[1] "Computing square of 'x'"
```

```
[1] 4
```

```r
memo_sq(2)
```

```
[1] 4
```

Caching: Rmarkdown

```
```{r import-data, cache=TRUE}
d <- read.csv('my-precious.csv')
```


```{r analysis, dependson='import-data', cache=TRUE}
summary(d)
```
```

- Requires explicit dependencies or changes not detected.
- Changes to functions or packages not detected.
- Good practice to frequently clear cache to avoid problems.
- targets is a better solution

Caching: Quarto

```
```{r}
#| label: import-data
#| cache: true
d <- read.csv('my-precious.csv')
```


```{r}
#| label: analysis
#| dependson: import-data
#| cache: true
summary(d)
```
```

- Same problems as Rmarkdown

- targets is a better solution